Progetto crittografia
“Secret messaging app”

Eric Aquilotti

21 marzo 2026

Indice

1 Analisi **3**
1.1 Introduzione . . . 3
1.1.1 Obiettivo . . . 3
1.1.2 Interazioni . . . 3
1.2 Requisiti . . . 3
1.3 Tipologie di messaggi . . . 4

2 Design **5**
2.1 Schermata utente . . . 5
2.1.1 Login . . . 5
2.1.2 Homepage . . . 5
2.1.3 Dettaglio chat . . . 5
2.2 Database . . . 5

3 Design Dettagliato **6**
3.1 Tecnologie utilizzate . . . 6
3.1.1 Linguaggio . . . 6
3.2 Cifratura . . . 7
3.2.1 Dettaglio . . . 7
3.3 Interazioni . . . 9
3.3.1 Tipologia di messaggi . . . 10
3.3.2 Protocollo scambio di chiavi . . . 11
3.4 Database . . . 16
3.4.1 Tecnologie . . . 16
3.4.2 Server . . . 17
3.4.3 Client . . . 18
3.5 Server . . . 19
3.5.1 Message router . . . 19
3.5.2 Connection Handler . . . 20
3.6 Client . . . 21
3.6.1 Server Connection . . . 21

3.6.2 Client 21
3.7 Configurazione e Costanti 22

4 Testing **23**
4.1 Cifratura 23
4.2 Altro 23
4.2.1 Handshake 23
4.2.2 Connection Handler 23
4.2.3 Message router 23
4.2.4 Test di accesso non autorizzato 24

5 Conclusione **25**
5.1 Impressioni sul progetto 25
5.2 Sviluppi futuri 25
5.2.1 Algoritmi di cifratura 25
5.2.2 Comunicazione tra client 26
5.2.3 Linguaggio 26
5.2.4 UI 26
5.2.5 Funzionalita' 26

Capitolo 1

Analisi

1.1 Introduzione

1.1.1 Obiettivo

Si vuole realizzare un'applicazione di messaggistica sicura. L'applicazione dovra' gestire un server che fa da tramite e si occupa della ricezione e invio dei messaggi agli utenti che vogliono comunicare. In questo documento spieghero' nel dettaglio il design, l'implementazione e le motivazioni delle scelte fatte riguardo le tecnologie utilizzate e gli algoritmi implementati.

1.1.2 Interazioni

Ci sono 2 tipi di entita', client e server. Ogni client potra' inviare diverse tipologie di messaggi al server che indicheranno le azioni da fare, ad esempio la registrazione di un nuovo utente, l'invio di un messaggio, lo scambio di chiavi e diverse altre che vedremo nel dettaglio piu' avanti.

1.2 Requisiti

L'applicazione dovra' garantire:

- Confidenzialita': nessuno esterno alla comunicazione deve essere in grado di leggere il contenuto dei messaggi scambiati.
- Integrita': vogliamo garantire che tutti i messaggi arrivino correttamente.
- Identificazione: il server deve essere sicuro dell'identita' dell'utente e viceversa.

1.3 Tipologie di messaggi

Ci sono diverse tipologie di messaggi che l'utente puo' mandare al server e vice versa, in questa sezione tratteremo i tipi di messaggi che l'utente puo' vedere e decidere di inviare, i messaggi che sono under the hood saranno spiegati nel dettaglio del design e dell'implementazione.

- Register [id, password]: il messaggio di register permette di registrare un nuovo utente con un id univoco e una password. A questo messaggio corrisponde una risposta del server con conferma di esecuzione oppure con rifiuto piu' tipo di errore.
- Login [id, password]: il messaggio di login permette di accedere a un account dato l'id e la password, anche in questo caso il server rispondera' con un messaggio di successo oppure di fallimento con relativo errore.
- Send message [id, message]: permette di inviare un messaggio all'utente con l'id specificato.

I messaggi che il server invia ai client sono tutte risposte a comandi ricevuti oppure servono per gli scambi di chiavi che saranno approfonditi successivamente.

Capitolo 2

Design

2.1 Schermata utente

2.1.1 Login

Nella schermata di login sara' possibile accedere tramite id e password oppure si potra' creare un nuovo utente scegliendo l'id e la password. Una volta eseguito il login si accedera' all'home page.

2.1.2 Homepage

Nella pagina principale sara' possibile visualizzare tutte le chat passate ed eseguire le seguenti azioni:

- Selezionare una chat e aprire la schermata di dettaglio chat.
- Creare una nuova chat con un utente inserendone l'id.

2.1.3 Dettaglio chat

Nella schermata di dettaglio di una chat sara' possibile visualizzare lo storico dei messaggi e ovviamente sara' possibile inviarne di nuovi.

2.2 Database

Il server avra' un database interno che contiene tutti gli utenti con relative password. Il server manterra' in memoria solo i messaggi non ancora richiesti mentre i client manterranno in un loro db locale tutte le conversazioni. Il client inoltre avra' un database in cui vengono mantenute le chiavi private scambiate con gli altri utenti o con il server.

Capitolo 3

Design Dettagliato

In questo capitolo andremo piu' a basso livello elencando le tecnologie che saranno utilizzate e definiremo meglio la struttura del db, interazioni e degli algoritmi di cifratura.

3.1 Tecnologie utilizzate

3.1.1 Linguaggio

Ho deciso di fare tutto in Python per diversi motivi:

- Gestione automatica dei BigInt, questo mi permette di gestire facilmente le cifrature asimmetriche.
- Facile creazione e modifica di database.
- Facile utilizzo di TCP socket.
- Conoscenza pregressa.

Sicuramente questa scelta ha molti contro che avrei valutato se fosse un progetto di lavoro o che comunque dovesse diventare di dominio e utilizzo pubblico. In particolare ovviamente il punto dolente e' la lentezza di esecuzione.

Nonostante Python abbia una fantastica libreria (Pycryptodome) ho deciso di non utilizzarla ma di implementare a mano tutti gli algoritmi. Questo e' il motivo principale che mi ha portato a preferire algoritmi meno sicuri ma piu' semplici da implementare.

3.2 Cifratura

Nello scambio di chiavi con il server sara' utilizzata una cifratura ibrida che unisce Diffie Hellman e RSA. Ho scelto di fare in questo modo perche' non volevo che il sistema fosse vulnerabile al MITM attack. Infatti se usassi solo diffie hellman potrei avere problemi se un attaccante si immettesse nella comunicazione. Invece utilizzando anche RSA per firmare i messaggi evito questo problema e inoltre garantisco forward secrecy in caso la chiave RSA fosse compromessa. Una volta scambiate le chiavi private la cifratura avverra' tramite DES che pur non essendo sicuro e' piu' semplice da implementare visto che ho deciso di non usare librerie crittografiche.

3.2.1 Dettaglio

RSA

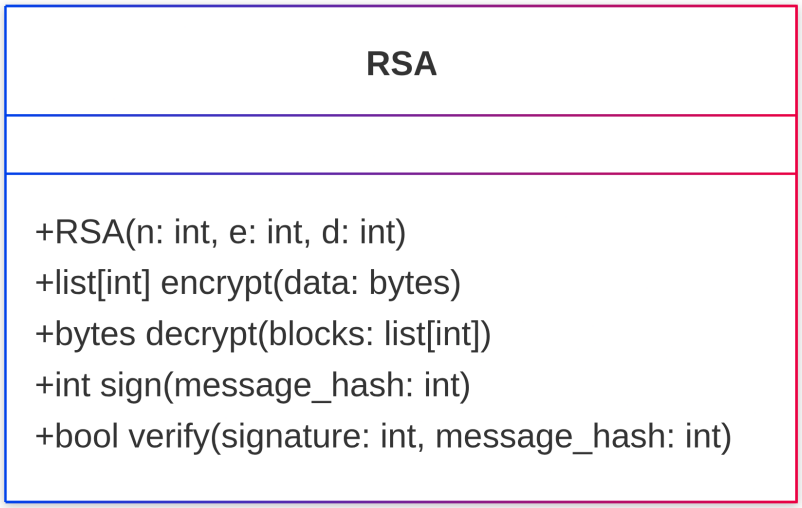

Figura 3.1: Diagramma della classe RSA.

Come si vede in 3.1, la classe RSA e' molto semplice e contiene le funzioni che ci serviranno, quindi cifratura, decifratura, firma e verifica della firma. Per RSA verranno utilizzate chiavi a 2048 bit. La chiave del server e' nota a priori ai client per evitare l'utilizzo di certificati.

Diffie Hellman

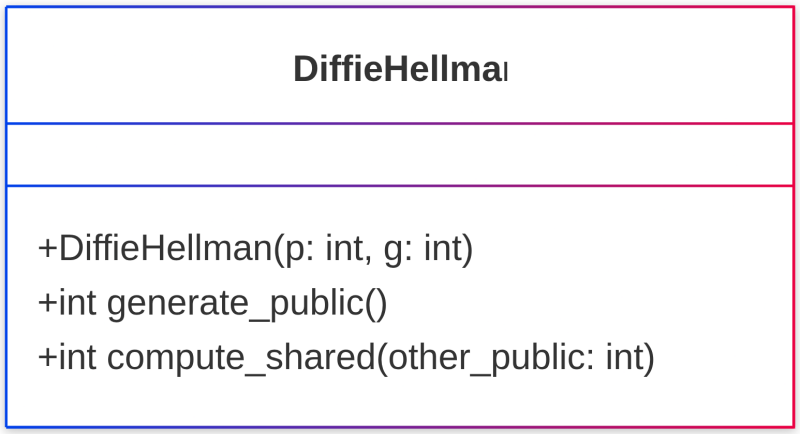

Figura 3.2: Diagramma della classe Diffie Hellman.

Anche in questo caso la classe e' molto semplice e contiene le funzioni per generare la chiave pubblica e ottenere la chiave condivisa. Per Diffie Hellman verranno utilizzate chiavi a 2048 bit. I valori di g e p sono fissati per ogni scambio.

Generazione numeri primi

I numeri primi saranno generati sfruttando il teorema di Fermat visto a lezione, quindi prendendo un numero casuale e testandone la primalita' in modo probabilistico. A lezione abbiamo visto che con un numero di 100 cifre la probabilita' di sbagliare e' minuscola e le chiavi a 2048 bit hanno circa 600 cifre quindi siamo abbastanza tranquilli. Non usero' il test di primalita' probabilistico esteso perche' non lo trovo necessario in questo caso avendo numeri molto grandi. Per la generazione di numeri random grandi usero' getrandbits della libreria secrets che usa un algoritmo sicuro crittograficamente.

DES

Figura 3.3: Diagramma della classe DES.

La classe DES permette, data la chiave, di cifrare e decifrare i messaggi. Per l'implementazione mi sono basato sulle indicazioni date qui e le tabelle sono state prese qui.

3.3 Interazioni

Per la comunicazione usero' il protocollo TCP con la libreria socket in quanto voglio avere la garanzia dell'arrivo corretto dei messaggi.

3.3.1 Tipologia di messaggi

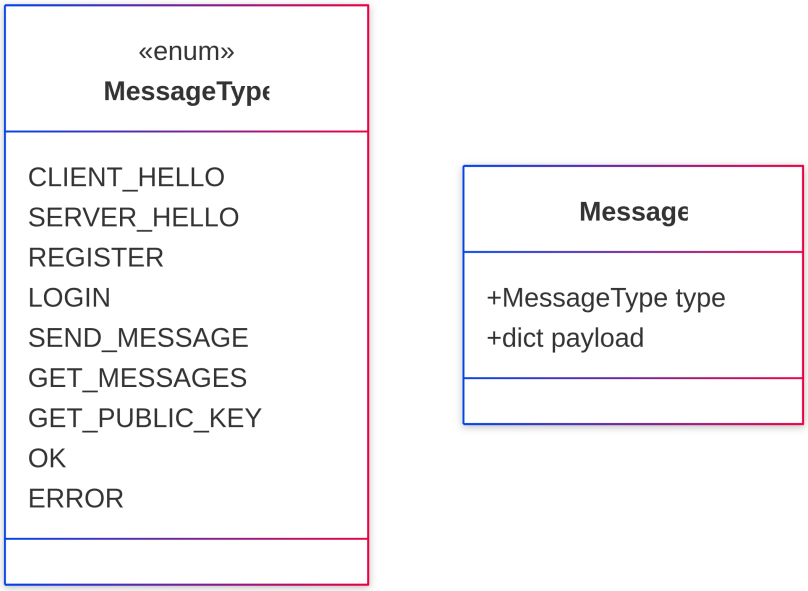

Figura 3.4: Diagramma della classe Message e MessageType.

Come si vede in 3.4 ci sono diversi tipi di messaggi:

- CLIENT_HELLO: il primo messaggio che il client manda al server per iniziare l'handshake.
- SERVER_HELLO: la risposta del server che conclude lo scambio di chiavi.
- REGISTER: inviato dal client per registrare un nuovo utente fornendo username e password.
- LOGIN: inviato dal client per eseguire il login fornendo username e password.
- SEND_MESSAGE: usato per l'invio di un messaggio client-client.
- GET_MESSAGES: usato per ottenere dal server tutti i messaggi non ancora ricevuti.
- GET_PUBLIC_KEY: usato dal client per ottenere la public key di qualcun altro dato l'username.

- OK: usato come risposta del server per confermare la corretta esecuzione di un'azione.
- ERROR: usato come risposta del server per segnalare un errore durante l'esecuzione di un'azione.

In 3.4 si vede anche la semplice struttura del messaggio che conterra' il tipo e il contenuto come dizionario.

3.3.2 Protocollo scambio di chiavi

In questa sezione vedremo la tipologia messaggi scambiati per le comunicazioni.

Scambio tra client e server

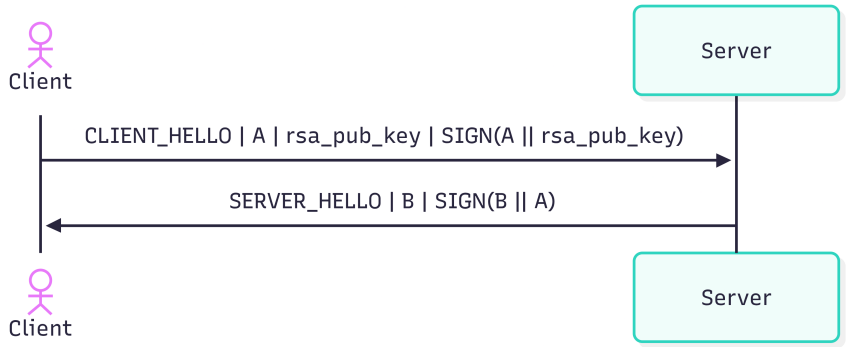

Figura 3.5: Diagramma di sequenza messaggi HELLO.

Come si vede in 3.5:

- Il client invia un messaggio CLIENT_HELLO con la chiave pubblica A per DH, la chiave pubblica RSA e la firma di tutto il payload.
- Il server risponde con SERVER_HELLO con l'altra parte di chiave B per DH e la firma. La firma ovviamente contiene anche A perche' il client deve essere sicuro della corretta ricezione da parte del server.
- Il client verifica la firma ed entrambi calcolano la chiave condivisa e puo' iniziare la comunicazione cifrata con DES.

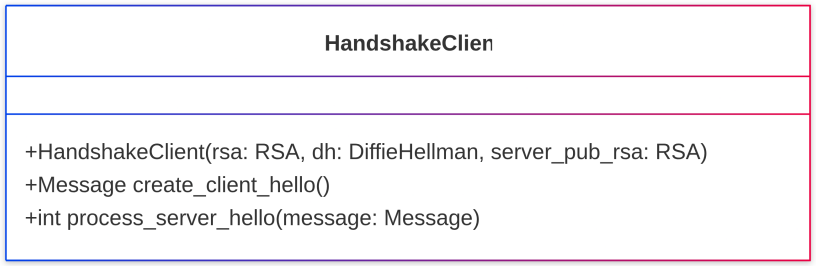

Figura 3.6: Diagramma della classe HandshakeClient.

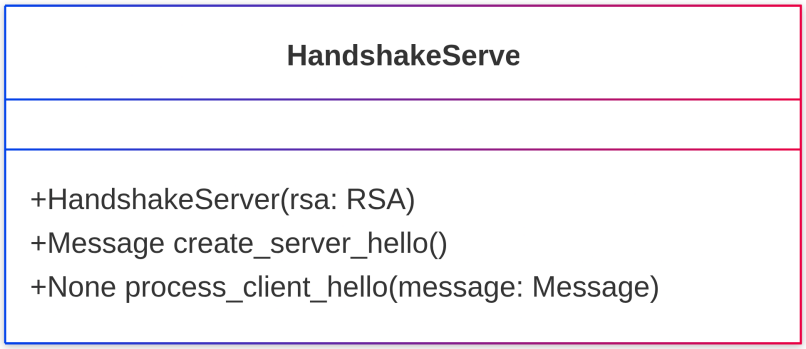

Figura 3.7: Diagramma della classe HandshakeServer.

Le classi 3.6 e 3.7 servono proprio per gestire lo scambio di chiavi usando DiffieHellman e verificando le firme.

Registrazione

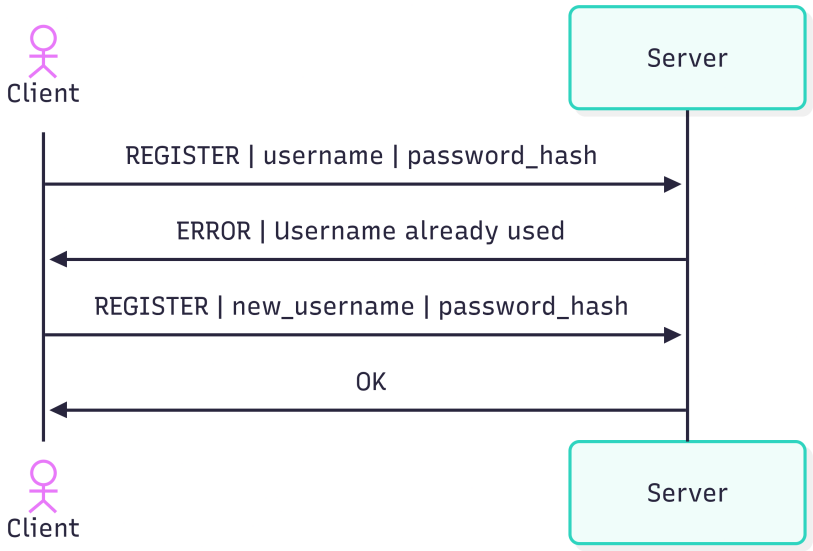

Figura 3.8: Diagramma di sequenza REGISTER.

Come si vede in 3.8, una volta scambiate le chiavi il client invia il messaggio REGISTER con username e password_hash. Il server salvera' tutti i dati dopo aver controllato che l'username sia univoco e poi rispondera' con OK oppure con ERROR con l'errore riscontrato.

Login

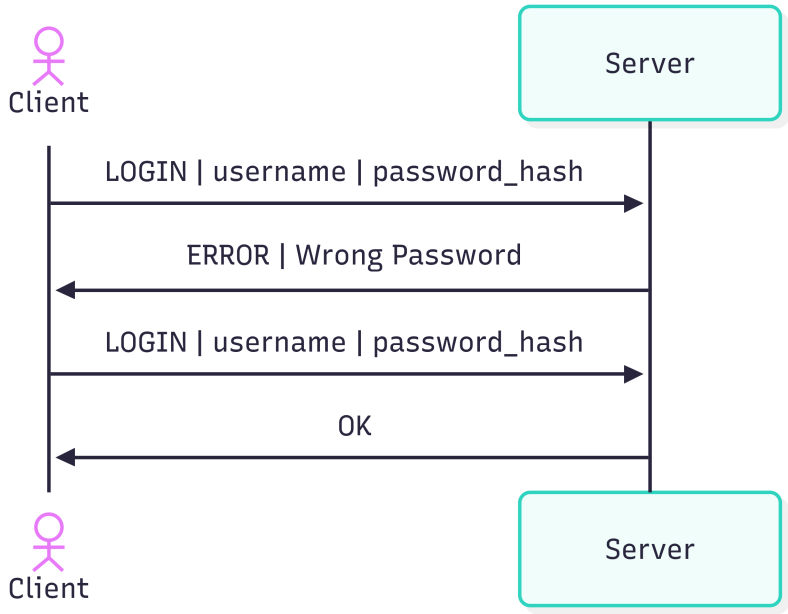

Figura 3.9: Diagramma di sequenza LOGIN.

Come si vede in 3.9, per il login il client inviera' un messaggio LOGIN con username e password_hash e il server rispondera' con OK oppure con ERROR con l'errore riscontrato.

Comunicazione tra client

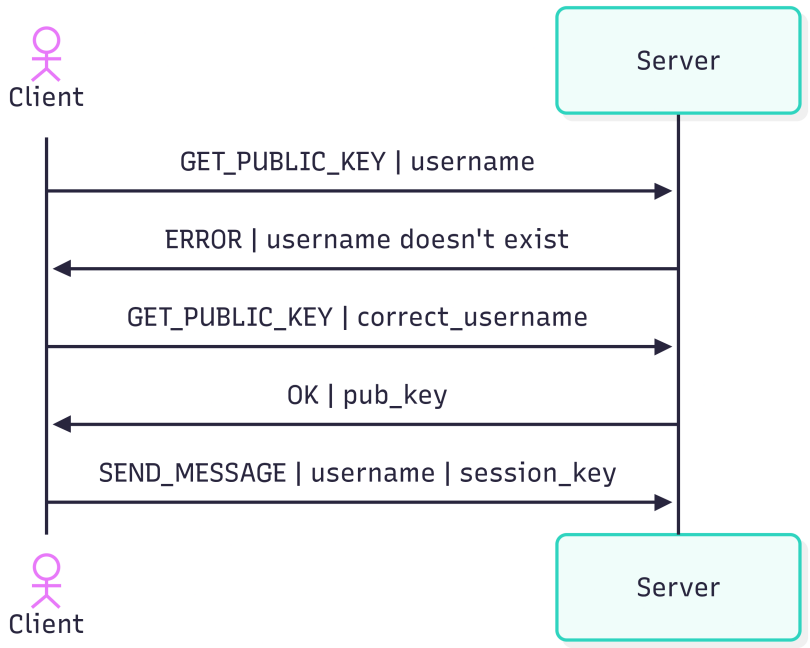

Figura 3.10: Diagramma di sequenza GET_PUBLIC_KEY.

Come si vede in 3.10, per la comunicazione tra client il server agisce solo come tramite dei messaggi. Il client inviera' un messaggio GET_PUBLIC_KEY con l'username dell'utente interessato e il server rispondera' con un messaggio OK e la chiave pubblica dell'utente oppure con ERROR se c'e' stato qualche errore. Successivamente il client genera una chiave simmetrica e la invia cifrata con la chiave pubblica del destinatario nel messaggio SEND_MESSAGE che contiene il destinatario e la chiave simmetrica cifrata.

Invio e richiesta messaggi

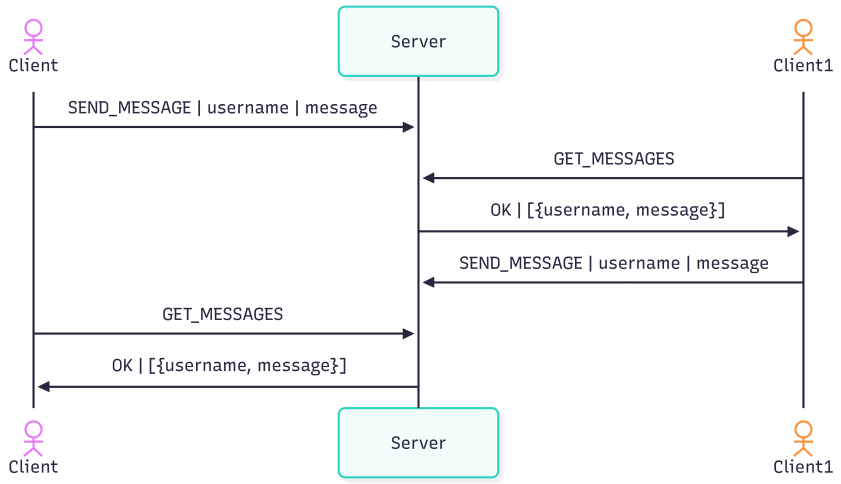

Figura 3.11: Diagramma di sequenza invio e richiesta messaggi.

Come si vede in 3.11, una volta eseguito il tutto i messaggi saranno della forma SEND_MESSAGE con destinatario e messaggio cifrato tramite DES. I client periodicamente inviano un messaggio GET_MESSAGES al server che rispondera' con un messaggio OK contentente tutti i messaggi nuovi.

3.4 Database

3.4.1 Tecnologie

Per i database ho deciso di utilizzare semplici file .db con la libreria sqlite3, in questo modo ho tutto integrato in Python ed e' piu' semplice l'implementazione.

3.4.2 Server

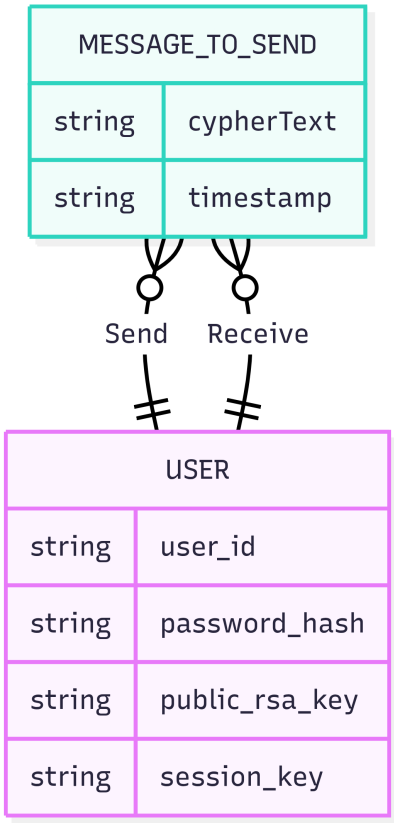

Figura 3.12: Diagramma er database del server.

Come si vede in 3.12 abbiamo 2 entita', una per gli utenti che sono identificati dall'id e contengono l'hash della password e le eventuali chiavi di sessione oppure pubbliche RSA. Ovviamente teniamo l'hash e non la password in chiaro perche' non vogliamo che il server conosca la password degli utenti. Il server manterra' anche i messaggi che non sono stati ancora ricevuti dai client e di questi ci interessa mittente, destinatario e ovviamente il messaggio cifrato con la chiave privata scambiata dagli utenti in modo che il server non possa leggere i messaggi ma faccia solo da tramite. Una volta ricevuto il comando GET_MESSAGES il server eliminera' dal database i messaggi.

Password Hashing

Ogni password subira' 2 hashing. Uno lato client per fare in modo che nella comunicazione non passi la password in chiaro. Per fare questo utilizzo

sha256 con un salt che e' fisso ed e' l'username. In questo modo vengono nascoste le password identiche ad eventuali attaccanti e inoltre ogni attaccante dovrebbe creare tabelle diverse per ogni utente se volesse cercare di ottenere le password.

Lato server viene eseguito un ulteriore hashing utilizzando PBKDF2-HMAC-SHA256 con salt casuale e 100 000 iterazioni. In questo modo risulta molto difficile, anche avendo il db, trovare le password velocemente perche' la computazione dell'hash non puo' essere fatta in modo rapido come un banale sha256. Ho deciso di non utilizzare Argon2 pur essendo piu' moderno perche' avrei docuto installare la libreria apposita, invece PBKDF2 e' gia' presente nella libreria hashlib.

3.4.3 Client

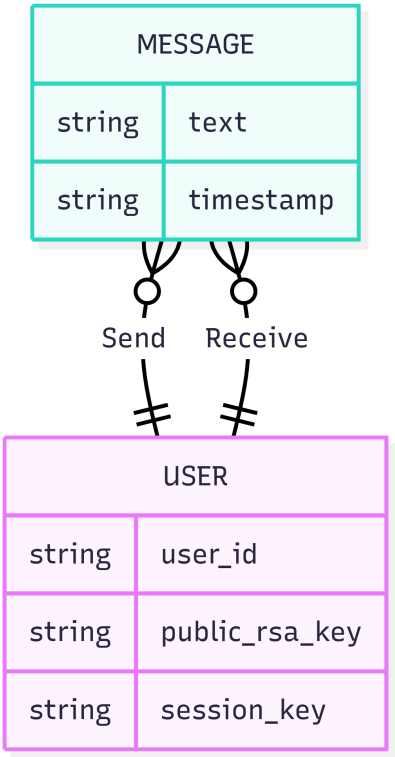

Figura 3.13: Diagramma er database del client.

Come si vede in 3.13 il diagramma per i client e' molto simile a quello del server. Le entita' sono le stesse ma cambiano i dati contenuti, in particolare per gli USERS non abbiamo l'hash della password per ovvi motivi e nei

MESSAGES manteniamo il messaggio in chiaro per evitare continue decifrature al caricamento del db. Possiamo mantenere il testo in chiaro perche' se un malintenzionato entra nel nostro PC avrebbe comunque accesso alle chiavi private e alle chiavi di sessione con gli altri utenti quindi sarebbe tutto inutile.

Il client avra' inoltre una tabella local_identity che contiene le cose personali quindi username e chiavi pubbliche e private di RSA.

3.5 Server

In questa sezione vedremo il design del funzionamento del server.

3.5.1 Message router

Per cercare di rendere il codice pulito ho deciso di scrivere una classe router che permette di registrare degli handler per specifiche tipologie di messaggi.

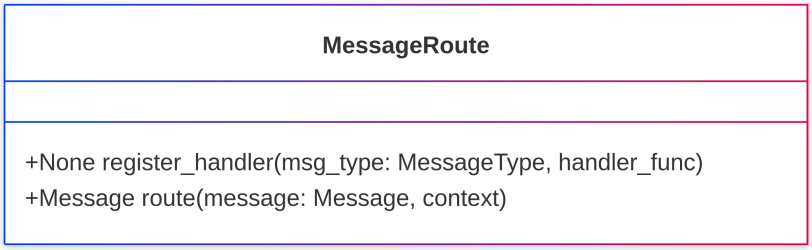

Figura 3.14: Diagramma della classe MessageRouter.

Come si vede in 3.14 c'e' la possibilita' di registrare gli handlers e di chiamare la funzione route che in base al context e al tipo di messaggio restituira' il messaggio di risposta.

Handlers

Gli handlers sono semplici funzioni che preso un messaggio e un context eseguono l'azione richiesta dai client. Ho definito i seguenti handlers:

- register_handler: gestisce l'azione di register di un nuovo user aggiungendolo al db verificando l'unicita' dell'username.
- login_handler: gestisce il login degli utenti aggiornando il context.

- get_messages_handler: gestisce le richieste dei messaggi di tipo GET_MESSAGES verificando che non ci siano accessi non autorizzati a messaggi.
- get_public_key_handler: gestisce le richieste di tipo GET_PUBLIC_KEY.
- send_message_handler: gestisce le richieste di tipo SEND_MESSAGE verificando che il sender sia effettivamente l'utente che ha fatto il login per evitare che qualcuno possa mandare messaggi per conto di altri.

Ogni handler restituisce un messaggio di risposta.

3.5.2 Connection Handler

Il ConnectionHandler e' colui che mantiene le connessioni con i vari client ed e' creato ad ogni nuova connessione alla socket. Per ogni connessione c'e' un Thread che viene avviato sulla funzione handle del ConnectionHandler, in questo modo posso facilmente gestire le sessioni perche' ogni Thread ne rappresenta una e agisce in autonomia rispetto agli altri.

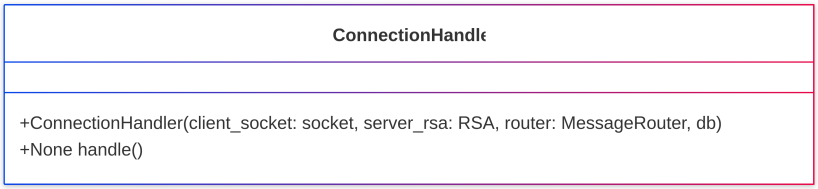

Figura 3.15: Diagramma della classe ConnectionHandler.

Come si vede in 3.16 vi e' una funzione handle su cui partira' il Thread e questa funzione eseguira' l'handshake e successivamente iniziera' il message loop in cui si attendono i messaggi e si eseguono le rispettive azioni utilizzando il MessageRouter.

3.6 Client

3.6.1 Server Connection

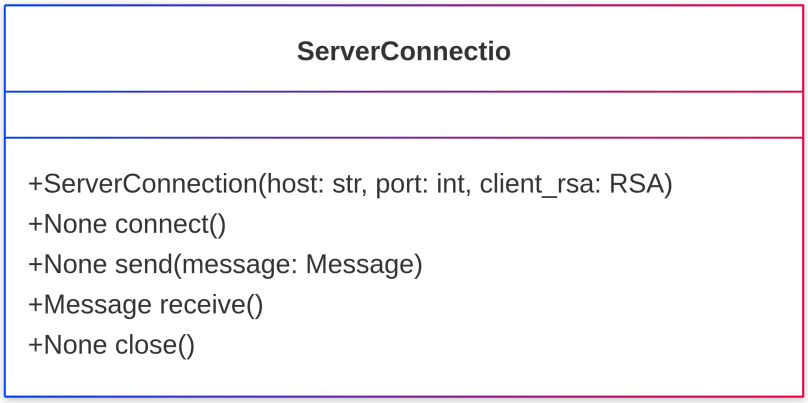

Figura 3.16: Diagramma della classe ServerConnection.

La classe ServerConnection si occupa di mantenere la connessione TCP col server e si occupa dell'invio e della ricezione dei messaggi in rete.

3.6.2 Client

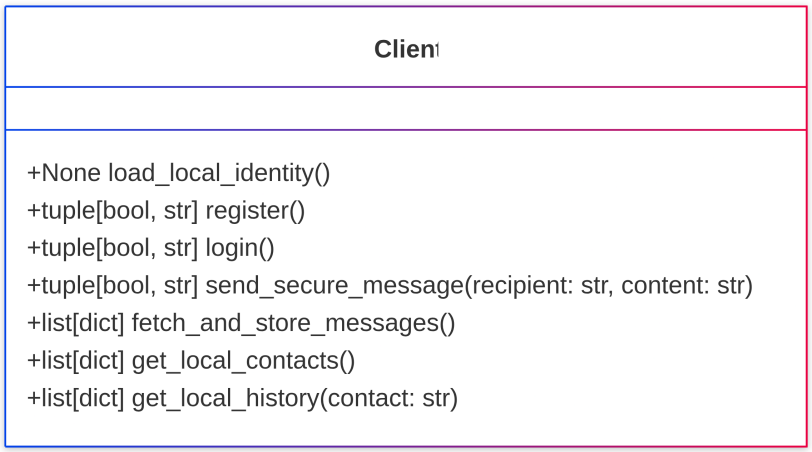

Figura 3.17: Diagramma della classe Client.

La classe Client permette di eseguire tutte le istruzioni necessarie agli utenti come registrarsi, eseguire il login, inviare messaggi ecc. Per l'invio dei messaggi al server utilizza una ServerConnection.

3.7 Configurazione e Costanti

Come detto in precedenza la chiave pubblica del server e' nota a priori a tutti i client. Anche per DH, p e g sono comuni a tutti ed ho deciso di utilizzare i valori forniti qui per 2048 bit.

Capitolo 4

Testing

In questa sezione vedremo quali test ho eseguito per verificare la correttezza degli algoritmi. Per implementare i test ho deciso di utilizzare la libreria unittest di python con eventualmente dei mock presi sempre dalla libreria unittest.

4.1 Cifratura

Per quanto riguarda la cifratura ho deciso di eseguire unit tests su tutti gli algoritmi implementati. In particolare ho eseguito test di cifratura e decifratura per DES e RSA. Inoltre ho testato la verifica della firma per RSA e il corretto scambio di chiavi per DH.

4.2 Altro

4.2.1 Handshake

Ho eseguito unittest anche per quanto riguarda Client e Server Handshake per verificare il corretto scambio delle chiavi e la corretta verifica della firma.

4.2.2 Connection Handler

Ho deciso di testare il ConnectionHandler per verificare il corretto invio e ricezione di messaggi e anche il corretto scambio di chiavi tramite handshake.

4.2.3 Message router

Ovviamente ho testato anche il router per verificare il corretto routing.

4.2.4 Test di accesso non autorizzato

Fortunatamente ho pensato all'idea di manomettere l'invio dei messaggi richiedendo messaggi di altri utenti o inviando messaggi facendo finta di essere altri senza aver fatto il login. Una volta testato mi sono accorto che il mio codice aveva un bug perche' non manteneva correttamente le sessioni e non veniva controllato che il login doveva essere fatto dall'utente di cui si richiedevano i messaggi. Quindi e' presente anche un test per controllare gli accessi non autorizzati.

Capitolo 5

Conclusione

5.1 Impressioni sul progetto

Ho sempre voluto scrivere un progetto del genere perche' ero interessato a capire veramente il funzionamento degli applicativi che utilizziamo tutti i giorni come WhatsApp che trovo molto interessanti. Mi sono divertito a cercare gli algoritmi e imparare le implementazioni. Sicuramente la parte piu' difficile e' stato lo strutturare bene il codice e cercare di renderlo pulito. In generale sono soddisfatto del prodotto ottenuto ma sono consapevole che non sia un prodotto finito ma che richieda ancora lavoro per essere considerato utilizzabile. E' stato un bel percorso molto interessante che mi ha permesso di imparare diverse cose tipo il discorso sugli hash delle password a cui non avevo pensato.

5.2 Sviluppi futuri

Ci sono moltissime cose da migliorare se volessi rendere questa applicazione veramente pubblica e utilizzabile.

5.2.1 Algoritmi di cifratura

Sicuramente la prima cosa da fare e' rendere veramente sicura la comunicazione implementando AES per la cifratura simmetrica e probabilmente Diffie Hellman sulle curve ellittiche per lo scambio delle chiavi. Inoltre si dovrebbe mettere la chiave RSA a 4096 bit. Un'altra cosa interessante da aggiungere sarebbe di cambiare le chiavi di sessione ogni N messaggi.

5.2.2 Comunicazione tra client

Al momento la comunicazione tra client non garantisce la forward secrecy perche' utilizzo RSA per cifrare la chiave. Potrei utilizzare la stessa implementazione di client e server ma questo implicherebbe che per inviare un messaggio a un nuovo utente egli debba essere online visto che Diffie Hellman richiede cooperazione. Una cosa realistica che si potrebbe implementare in futuro e' un sistema per cui ogni client carica sul server un certo numero di chiavi precalcolate e quando qualcuno vuole creare una comunicazione prende una di quelle chiavi usa e getta e si crea la chiave privata senza avere il bisogno che il ricevente sia online.

5.2.3 Linguaggio

Per ottenere la sicurezza dell'applicativo non e' necessario cambiare linguaggio ma sicuramente utilizzerei delle librerie gia' fatte che eseguono le istruzioni in C, ad esempio pycryptodome. Questo perche' mi rendo conto che il mio codice e' lentissimo a generare i numeri primi grandi e purtroppo questo accade principalmente perche' ho usato Python puro senza chiamate a C.

5.2.4 UI

Sicuramente un miglioramento alla UI e' necessario in quanto non le ho dato alcun peso durante il progetto, ho semplicemente usato la libreria tkinter per rendere l'implementazione piu' rapida mettendo qualche bottone e qualche textbox. Un ottimo update sarebbe di usare html, js e css per la UI.

5.2.5 Funzionalita'

Una volta creata una buona base si puo' pensare a integrare diverse funzionalita' come lo scambio di file e la creazione di gruppi.